

SQL Fundamentals

Note:

This is NOT a full SQL fundamentals course

Tables

Example `stock` table:

stock_id	name	organism_id
81	IR 62266-42-6-2	9
115	CARREON	9
426	PSBRC 50::IRGC 99706-1	9

`Rows` = individual records

`Columns` = attributes

Relational Databases

example `organism` table:

organism_id	genus	species	common_name
9	Oryza	sativa japonica nipponbare	Japonica nipponbare

example `stock` table:

stock_id	name	organism_id
81	IR 62266-42-6-2	9
115	CARREON	9
426	PSBRC 50::IRGC 99706-1	9

■ Primary key

uniquely
identifies each
row.

■ Foreign key

links to a row
in another
table.

Connecting to PostgreSQL

We've set up a database for you!

Parameter	Value
Database name	rice163
Username	trainee
Hostname	10.0.9.54
Port	5433

```
psql -d {database_name} -U {username} -h {hostname} -p {port}
```

```
psql -d rice163 -U trainee -h 10.0.9.54 -p 5433
```

Some PostgreSQL Commands

Command	Description
<code>\dt</code>	List tables in the current database
<code>\d {table_name}</code>	Describe a table's columns and types
<code>\q</code>	Quit psql

Get Rows

```
SELECT * FROM {table};
```

```
SELECT * FROM organism;
```

To allow sideways scrolling for wide tables: `\setenv PAGER 'less -S'`

 Do not forget the semicolon (;)

Capitalization is **NOT** required for SQL keywords.

Get rows - with specific columns

```
SELECT {column1}, {column2}, {...} FROM {table};
```

```
SELECT genus, species FROM organism;
```

Get rows – with a specific number of rows

Using `LIMIT`

```
SELECT {columns...} FROM {table} LIMIT {row_count};
```

```
SELECT * FROM feature LIMIT 5;
```

When exploring, add `LIMIT` unless you need every row. Make it a habit.

Counting Rows

```
SELECT COUNT(*) FROM {table};
```

```
SELECT COUNT(*) FROM feature;
```

Practice (4 min)

Before starting, do `\setenv PAGER 'less -S'`

1. How many rows are there in `stock` table?
2. View 5 rows from the `stock` table showing all columns.
3. View 5 rows from the `stock` table showing only the `name` column.

Input	Meaning
<code>SELECT *</code>	Show all columns
<code>SELECT {column}</code>	Show one column
<code>FROM {table}</code>	Choose table
<code>COUNT(*)</code>	Count rows
<code>LIMIT {n}</code>	Show only n rows

Filtering Rows

Using `WHERE`

```
SELECT {columns...} FROM {table}  
WHERE {condition};
```

```
SELECT * FROM stock  
WHERE name = 'CARREON';
```

Operator	Meaning
=	Equal to
!=, <>	Not equal to
>, <	Greater / less than
>=, <=	Greater / less than or equal
LIKE	Pattern match (% as wildcard)
IS NULL	Value is missing
IN (...)	One value from a list

Filtering Rows

■ Pattern match with `LIKE`

```
SELECT name FROM stock
WHERE name LIKE '%G1'
LIMIT 10;
```

`%` matches any sequence of characters.

Filtering Rows

■ Combine conditions

```
SELECT organism_id, position, refcall FROM snp_featureloc
WHERE position >= 1000
      AND position <= 2000
LIMIT 10;
```

Practice (8 min)

1. Find 10 rows from `snp_featureloc` where `position` is greater than `50000`.
2. Find 10 `stock` rows where the `name` starts with `IR`.
3. Find all `db` rows where `description` is `NULL`.

Input	Meaning
<code>WHERE</code>	Filter rows
<code>></code>	Greater than
<code>=</code>	Equal to
<code>LIKE 'x%'</code>	Starts with <code>x</code>
<code>IS NULL</code>	Value is missing
<code>LIMIT {n}</code>	Show only n rows

Sorting Rows

Using `ORDER BY`

```
SELECT {columns...} FROM {table}  
ORDER BY {column} {ASC|DESC}  
LIMIT 10;
```

```
SELECT name FROM stock  
ORDER BY name DESC  
LIMIT 10;
```

INNER JOIN

Returns rows where the condition matches in both tables.

Using `JOIN` and `ON`

```
SELECT o.common_name, s.name AS stock_name
FROM organism o
JOIN stock s
ON s.organism_id = o.organism_id
LIMIT 10;
```

Use aliases to avoid ambiguous column names.

Example: `o.name` vs `s.name`

Joining Multiple Tables

```
SELECT f.name AS feature_name,  
       cv.name AS vocabulary,  
       ct.name AS term  
FROM feature f  
JOIN cvterm ct ON f.type_id = ct.cvterm_id  
JOIN cv ON ct.cv_id = cv.cv_id  
LIMIT 20;
```

Practice (8 min)

1. Sort `snp_featureloc` by position. Limit to 10 rows.
2. Join `snp_featureloc` and `feature`. Show `refcall` and `position` for each SNP. Limit to 10 rows.

Use:

```
snp_featureloc.srcfeature_id = feature.feature_id
```

Input	Meaning
<code>ORDER BY</code>	Sort rows
<code>JOIN</code>	Matching rows only
<code>ON</code>	Join condition
<code>table.column</code>	Specific column
<code>AS</code>	Rename output column
<code>WHERE right_table.id IS NULL</code>	Rows with no match
<code>LIMIT {n}</code>	Show only n rows

Query Structure

Order matters:

1. SELECT
2. FROM
3. JOIN
4. WHERE
5. ORDER BY
6. LIMIT

Common Mistakes

1. Forgetting `;`
2. Forgetting `LIMIT` when exploring large tables
3. Using `= NULL` instead of `IS NULL`
4. Putting clauses in the wrong order, e.g. `LIMIT` before `WHERE`
5. Forgetting quotes around text values, e.g. `name = CARREON`
6. Misusing `LIKE` wildcards, e.g. `LIKE 'IR'` instead of `LIKE 'IR%'`
7. Ambiguous column names after joins, e.g. `name` instead of `stock.name` or an alias

What To Learn Next

- Other types of JOINS
- INSERT, UPDATE, DELETE
- \COPY
- Subqueries
- Indexes
- Transactions